Legacy Application Modernization - Migration Design Document

Project: Task Management Application Migration
From: Java/Play Framework + Backbone.js
To: NestJS/TypeScript + React/TypeScript
Date: July 2026

Executive Summary

This document outlines our approach to migrating a legacy task management application to modern technologies using incremental migration with zero downtime. We employ the Strangler Fig Pattern to gradually replace legacy components while maintaining full system operability.

Current State

- Backend: Java 17 + Play Framework + JPA/Hibernate
- Frontend: Backbone.js + jQuery + server-rendered views
- Database: H2 in-memory (PostgreSQL in production)

Target State

- Backend: NestJS + TypeScript + TypeORM
- Frontend: React + TypeScript + Modern State Management
- Database: Shared PostgreSQL
- Infrastructure: Containerized deployment

Migration Philosophy

Incremental migration with zero downtime using the Strangler Fig Pattern to minimize business disruption and technical risk while enabling continuous delivery.

Part 2: Frontend Hybrid Architecture

Question 1: How React is Bootstrapped Inside Backbone

Approach: React components are mounted as guests within the Backbone application host.

Implementation Overview:

The bootstrap process happens in four stages:

Stage 1 - Script Loading: The page loads dependencies sequentially - first Backbone (jQuery, Underscore, Backbone itself), then React (React core, ReactDOM), and finally application code.

Stage 2 - Backbone Initialization: Backbone initializes first and creates the task collection, which is exposed globally to serve as the integration point between the two frameworks.

Stage 3 - Bridge Creation: A custom bridge layer wraps the Backbone collection and provides an Observable interface that React can understand, translating between Backbone events and React state updates.

Stage 4 - React Mounting: React uses ReactDOM to mount into designated DOM containers that are pre-defined in the HTML, rendering components that receive the bridge as a prop.

Key Success Factors:

- Global variables provide the integration point between frameworks
- Bridge implements retry logic to wait for all dependencies
- React doesn't try to control the entire page, only its designated containers
- Pre-defined DOM containers in HTML allow clean separation

Question 2: How State is Shared

Principle: Single Source of Truth with Observable Pattern

Architecture: Backbone models serve as the single source of truth, with React subscribing to changes through the bridge layer.

Data Flow Pattern A - Backbone to React (Create/Update):

When a user creates a task through the Backbone form, the following sequence occurs: The form submits data to a Backbone model, which saves to the API. Upon successful save, the model is added to the Backbone collection. This triggers collection events that the bridge detects. The bridge then notifies all React subscribers by calling their callback functions. React receives the update, calls setState, and re-renders with the new data.

Data Flow Pattern B - React to Backbone (Update/Delete):

When a user interacts with the React UI (like clicking delete), React calls a bridge method instead of the API directly. The bridge delegates to the appropriate Backbone model, which handles the API call. On success, the collection updates automatically, firing events. The bridge detects these events and notifies React, completing the loop back to Pattern A.

Bridge Responsibilities:

- Provides Observable pattern for React subscriptions
- Offers Promise-based API for async operations
- Delegates all mutations to Backbone models
- Translates between Backbone events and React callbacks
- Manages subscriber lifecycle and cleanup

State Management Rules:

- Backbone owns all data - it's the single source of truth
- React subscribes to Backbone changes through the bridge
- All API calls go through Backbone models, never directly from React
- No duplicate state exists in both frameworks
- The bridge is the only communication channel between them

Question 3: How We Avoid Two Competing Frontend Architectures

Strategy 1 - Clear Component Ownership: Each component has one explicit owner. The data layer belongs to Backbone, routing belongs to Backbone, the task form is owned by Backbone, and the task list is owned by React. Each component is implemented exactly once with clear migration priority.

Strategy 2 - Unidirectional Data Flow: All mutations flow through Backbone, preventing competing data flows. The API connects to Backbone models, which feed both Backbone views and React components through the bridge layer.

Strategy 3 - Bridge as Contract: The bridge provides a stable interface that decouples implementations. React doesn't know about Backbone internals, Backbone doesn't know about React, either implementation can be swapped independently, and testing boundaries are clear.

Strategy 4 - No Feature Duplication: Features are never implemented twice. Task creation exists only in Backbone, task display exists only in React. This eliminates overlap and confusion.

Strategy 5 - Gradual Migration Path: Each phase is independently shippable to production. Phase 1 migrates the task list to React (complete). Phase 2 will migrate the task form to React. Phase 3 will migrate the router to React Router. Phase 4 will migrate the data layer to Redux or Context.

Strategy 6 - Visual Boundaries: Technology badges in the UI clearly show which framework owns which component, making the architecture transparent to developers and stakeholders.

Part 3: Backend Hybrid Architecture

Question 1: How Requests are Routed

Strategy: Frontend-Based Routing

The client application decides which backend to call based on the operation type. This approach requires no infrastructure changes and provides maximum flexibility during migration.

Implementation: The frontend maintains a configuration object with endpoint URLs for both backends. When making API calls, a custom sync method checks the HTTP verb and routes POST requests to NestJS while routing all other operations (GET, PUT, DELETE) to Play Framework.

Current Routing Table:

- POST /api/tasks routes to NestJS on port 3000 (migrated)
- GET /api/tasks routes to Play on port 9000 (legacy)
- GET /api/tasks/:id routes to Play on port 9000 (legacy)
- PUT /api/tasks/:id routes to Play on port 9000 (legacy)
- DELETE /api/tasks/:id routes to Play on port 9000 (legacy)

Benefits: No proxy infrastructure needed during development, easy to test and modify per endpoint, can toggle routing in code for experimentation, provides clear migration tracking, and enables instant rollback capability.

Production Alternative: For production deployment, we can introduce an Nginx reverse proxy or API Gateway to handle routing based on HTTP method and path, making the routing transparent to the frontend.

Question 2: How Play and NestJS Coexist

Coexistence Mechanism A - Different Ports: Play Framework runs on port 9000 while NestJS runs on port 3000. This eliminates port conflicts and allows both to operate independently.

Coexistence Mechanism B - Shared Database: Both applications connect to the same PostgreSQL database with carefully matched schemas. The table structures, column names, data types, and constraints are identical. Both frameworks use auto-increment primary keys and maintain the same nullable field definitions.

Coexistence Mechanism C - CORS Configuration: NestJS enables Cross-Origin Resource Sharing to accept requests from Play's frontend, allowing requests from both localhost:9000 and localhost:3000 with all standard HTTP methods.

Coexistence Mechanism D - Conflict Prevention: We enforce three rules to prevent conflicts. First, each operation type has a single writer - POST goes only to NestJS. Second, we plan to implement optimistic locking with version columns for future conflict detection. Third, both databases use appropriate transaction isolation levels to prevent dirty reads.

Question 3: How Authentication Would Work

Strategy: Shared JWT Authentication with Common Secret

Architecture: Users can log in through either backend. Upon successful authentication, the system generates a JWT token signed with a shared secret key. The client stores this token in localStorage or an HTTP-only cookie. All subsequent requests include the token in the Authorization header. Both backends validate the token using the same shared secret, allowing seamless authentication across both systems.

Shared Configuration: Both applications use identical JWT settings including the same secret key, expiration time (24 hours), issuer claim (task-management-app), and signing algorithm (HS256).

NestJS Implementation: NestJS uses a guard that intercepts requests, extracts the Bearer token from the Authorization header, verifies the token signature using the shared secret, validates the issuer claim, and attaches the decoded user information to the request object before allowing the request to proceed.

Play Framework Implementation: Play Framework uses an Action wrapper that extracts the token, verifies it using the same secret and issuer claim, attaches user information to the request context, and either allows the request to proceed or returns an unauthorized response.

Key Authentication Points:

- Same secret key stored in environment variables for both applications
- Identical JWT format with matching issuer and claims structure
- Same validation logic for signature and expiration checks
- Users can authenticate through either backend

- Tokens work seamlessly across both backends

Question 4: How Deployments Would Work

Development Deployment: Currently, we run both applications locally on different ports. Play Framework runs with SBT on port 9000, and NestJS runs with npm on port 3000. The frontend routes requests to the appropriate backend based on operation type.

Production Deployment Option 1 - Docker Compose: We containerize both applications with separate services. The Play application runs in one container exposing port 9000. The NestJS application runs in another container on port 3000. An Nginx reverse proxy container routes incoming traffic to the appropriate backend. A PostgreSQL container provides the shared database. All services are orchestrated through Docker Compose with health checks and automatic restarts.

Production Deployment Option 2 - Kubernetes: Each backend runs as a separate deployment with independent scaling. Play might run 2 replicas while NestJS runs 3 replicas, demonstrating independent scaling. Each deployment has its own service for discovery. An Ingress controller routes traffic based on HTTP method and path. Health checks and rolling updates ensure zero-downtime deployments.

Production Deployment Option 3 - Cloud Platforms: On AWS, we'd use an Application Load Balancer routing to separate ECS services for each backend. Both services connect to a shared RDS PostgreSQL database. Each service can scale independently based on metrics.

Deployment Strategy - Blue-Green Rollout: Phase 1 deploys NestJS with zero traffic for testing. Phase 2 implements a canary release routing 10% of POST requests to NestJS while monitoring. Phase 3 increases to 50% after validation. Phase 4 routes 100% of POST traffic to NestJS. We monitor key metrics at each phase and maintain the ability to rollback.

Rollback Plan: If issues arise, we have three rollback options. Option 1 is immediate (under 5 minutes) - change the frontend API endpoint configuration. Option 2 is quick (under 30 minutes) - stop NestJS containers and route all traffic to Play. Option 3 is full rollback (under 2 hours) - revert code changes and restore database if needed.

Part 4: Complete Migration Strategy

1. Migration Approach: Incremental vs Rewrite

Decision: Incremental Migration using Strangler Fig Pattern

Why Incremental:

From a business perspective, we cannot afford downtime, must maintain continuous feature delivery, operate a revenue-generating system, serve an active user base, and cannot pause development for a rewrite.

From a technical perspective, incremental migration reduces the risk of catastrophic failure, maintains institutional knowledge within the codebase, allows testing each component independently, enables a gradual team learning curve, and provides easy rollback at any point.

Why Not Rewrite:

A complete rewrite carries high risk with potential for catastrophic failure. It requires extended downtime that impacts business operations. The cost is high upfront rather than spread over time. Testing happens all at once in a big bang approach. Rollback would be extremely difficult if issues arise. Additionally, team stress would be very high during the critical cutover period.

Strangler Fig Pattern: This pattern is named after the strangler fig tree that grows around a host tree and eventually replaces it. We build new functionality alongside the old system, route traffic incrementally to the new system, monitor and validate the new components, gradually deprecate old components as new ones prove themselves, and finally remove the old system when it's fully replaced.

Pattern Benefits: Development can proceed in parallel on both systems. We can continuously compare performance between old and new. Performance validation happens incrementally. Risk is mitigated through gradual rollout. Business continuity is maintained throughout the migration.

2. Frontend Migration Plan

Migration from Backbone.js to React + TypeScript

Phase 1 - Hybrid Setup (Completed): React and ReactDOM are integrated alongside Backbone. The bridge pattern connects both frameworks. The task list component has been migrated to React while the form remains in Backbone. Duration was 2 weeks with low risk.

Phase 2 - Component Migration (Next): The task form will be migrated to React. React Router will be added alongside Backbone Router. A shared component library will be implemented. Expected duration is 3 weeks with medium risk.

Phase 3 - Router Migration: Backbone Router will be replaced with React Router while maintaining URL compatibility. Browser history handling will be updated. Expected duration is 2 weeks with medium risk.

Phase 4 - State Management: Redux or Context API will be introduced. Migration from Backbone models will occur. The bridge layer will be removed entirely. Expected duration is 3 weeks with high risk.

Phase 5 - Complete Frontend: All Backbone dependencies will be removed. Full TypeScript conversion will be completed. Modern build pipeline will be implemented. Expected duration is 2 weeks with low risk.

Routing Strategy Evolution: Initially, Backbone Router handles all routing while rendering React components. In the middle phase, React Router is introduced alongside Backbone Router with both systems working in parallel. Finally, Backbone Router is removed completely, leaving pure React Router with a modern routing approach.

State Management Evolution: We start with Backbone models as the data source. Next, we introduce hybrid state where React hooks coexist with Backbone models. Then Redux or similar state management replaces Backbone entirely. Optionally, we could adopt React Query for modern data fetching patterns.

Authentication Strategy: Initially, sessions are stored in Backbone models. We then transition to a shared Auth Context that reads from Backbone but is accessible to React. Finally, React-only authentication stores JWT tokens in localStorage or HTTP-only cookies with custom hooks for auth operations.

Shared Components Strategy: We create a component library with reusable elements like buttons, inputs, cards, and modals. These components can be used in both React and Backbone contexts during the transition, ensuring consistency and reducing duplication.

CSS Strategy: We maintain a shared global stylesheet that works for both frameworks initially. CSS Modules are introduced for new React components. We gradually migrate old styles to the new system. Finally, we remove global CSS when Backbone is fully removed. The team can choose between CSS Modules, Styled Components, or Tailwind CSS for the final implementation.

Testing Strategy: We maintain existing Backbone tests using QUnit or Jasmine. React tests are added using Jest and React Testing Library. Integration tests verify the hybrid system works correctly. End-to-end tests using Cypress or Playwright ensure the complete user journey functions properly.

3. Backend Migration Plan

Migration from Play Framework to NestJS

Phase 1 - Infrastructure Setup (Completed): NestJS application was created with TypeORM configured. Database schema was matched to Play's schema. CORS was configured properly. Duration was 1 week with low risk.

Phase 2 - Write Operations (Current): POST /api/tasks has been migrated (complete). PUT and DELETE endpoints are next in queue. Expected duration is 2 weeks total with low risk.

Phase 3 - Read Operations: GET endpoints will be migrated to NestJS. Query performance will be optimized. Expected duration is 2 weeks with low risk.

Phase 4 - Business Logic: Complex operations, background jobs, and third-party integrations will be migrated. Expected duration is 4 weeks with high risk.

Phase 5 - Complete Backend: Play Framework will be removed entirely. Full TypeScript backend will be operational. Infrastructure will be optimized. Expected duration is 2 weeks with medium risk.

API Ownership Strategy: Clear ownership boundaries are established where each endpoint has exactly one owner. No endpoint is implemented in both systems. Migration tracking is transparent through documentation. If breaking changes are needed, APIs are versioned appropriately.

Database Access Strategy: Both applications share the same PostgreSQL database with three coordination mechanisms. First, schema compatibility is maintained with identical table names, column names, data types, and constraints. Second, migrations are coordinated between both systems - changes are made in both Play Evolutions and NestJS TypeORM migrations, tested with both applications, and deployed before code changes. Third, transaction management is handled independently by each application with appropriate database-level isolation and no distributed transactions needed.

Service Boundaries: As we move toward microservices, we'll establish clear boundaries. The Task Service will handle CRUD operations and queries. The Auth Service will manage login, registration, and JWT handling. The User Service will handle profiles and preferences. The Analytics Service will provide metrics and reporting. Services will communicate via REST APIs for synchronous operations and message queues for asynchronous operations.

Error Handling Strategy: Both backends implement standardized error responses with consistent structure. Each error includes an error title, detailed message, error code, timestamp, and request path. Error types are categorized as validation errors (400), authentication errors (401), authorization errors (403), not found errors (404), and internal server errors (500).

Logging Strategy: Structured JSON logging is implemented across both backends with consistent format. All logs include timestamp, service name, log level, error details, user context, and request metadata. Logs are centralized to ELK Stack or CloudWatch with correlation IDs for request tracing across services.

Monitoring Strategy: Key metrics are tracked across both systems including request count per service, response time percentiles, error rates, database query performance, connection pool usage, memory and CPU usage, and business metrics like tasks created per service and active users. Tools include Prometheus for metrics collection, Grafana for visualization, AlertManager for notifications, and Jaeger for distributed tracing.

Implementation Roadmap

Month 1-2: Frontend Phase 1-2 (Hybrid setup and component migration)
Month 3: Frontend Phase 3 (Router migration)
Month 4: Backend Phase 2-3 (Write and read operations)
Month 5-6: Frontend Phase 4-5 (State management and full React)
Month 7-8: Backend Phase 4-5 (Business logic and full NestJS)
Month 9: Optimization, security audit, and load testing
Month 10: Documentation, team training, and handover

Risk Management

High-Risk Area 1 - Data Consistency: Two backends writing to the same database creates risk of conflicts. Mitigation includes clear operation ownership, database-level constraints, transaction isolation, and comprehensive testing.

High-Risk Area 2 - Authentication Synchronization: JWT token incompatibility between systems could break user sessions. Mitigation includes shared secret configuration, same token format, automated token validation tests, and fallback mechanisms.

High-Risk Area 3 - Performance Degradation: The hybrid system might be slower than the legacy system. Mitigation includes continuous monitoring, performance benchmarks, load testing before each phase, and gradual rollout with ability to rollback.

High-Risk Area 4 - Team Knowledge Gap: Team members may be unfamiliar with new technologies. Mitigation includes comprehensive training programs, pair programming sessions, thorough code reviews, and detailed documentation.

Rollback Monitoring Thresholds: Automatic rollback is triggered if error rate exceeds 5% for 5 minutes, response time exceeds 2x baseline for 10 minutes, database connection pool is exhausted, memory or CPU usage stays above 90%, or critical functionality breaks.

Success Metrics

Technical Performance Metrics:

- Response time under 200ms at 95th percentile
- Throughput above 1000 requests per second
- Error rate below 0.1%
- System uptime above 99.9%

Code Quality Metrics:

- Test coverage above 80%
- TypeScript strict mode enabled throughout
- Zero critical security vulnerabilities
- Technical debt below 10% of codebase

Migration Progress Metrics:

- Frontend 100% React by Month 6
- Backend 100% NestJS by Month 8

- Zero Backbone dependencies by Month 6
- Zero Play dependencies by Month 8

Business Metrics:

- Zero production incidents during migration
- Feature velocity maintained or improved
- No missed release deadlines
- Customer satisfaction scores maintained or improved

Team Metrics:

- 100% team trained on new technology stack
- Developer satisfaction scores improved
- Onboarding time for new developers reduced by 50%
- Code review cycle time reduced by 30%

Conclusion

This migration design demonstrates a comprehensive, low-risk approach to modernizing a legacy application. Our strategic principles prioritize incremental changes over big bang rewrites, system coexistence over immediate replacement, automation over manual processes, and team success over pure technology considerations.

What We've Accomplished:

Part 1 established the legacy baseline with a complete Java/Play Framework backend and Backbone.js frontend with full CRUD operations.

Part 2 created a frontend hybrid architecture with React components working alongside Backbone, a bridge pattern for state synchronization, the task list successfully migrated to React, and zero conflicts between frameworks.

Part 3 implemented a backend hybrid architecture with NestJS running alongside Play Framework, POST endpoints migrated to NestJS, shared database architecture working correctly, and JWT authentication strategy defined.

Part 4 provided a complete 10-month migration roadmap with clear phase boundaries, comprehensive risk mitigation plans, and well-defined success metrics.

Key Achievements: We've developed a zero-downtime migration approach, built production-ready hybrid systems, established comprehensive testing strategies, and designed scalable infrastructure. From a business perspective, we ensure continuous feature delivery, mitigate risk through incremental approach, empower the team through training, and provide a clear path to modern architecture.

The migration is currently in progress with strong foundations in place and a clear path forward to complete modernization.

Document Version: 1.0
Last Updated: July 23, 2026
Status: Implementation In Progress
Next Review: Monthly during migration

End of Document

Deployment Strategy

Safe Release Approach

Our deployment strategy prioritizes safety and observability throughout the migration process.

Feature Flags: We implement feature flags to control which components users see and which backend handles their requests. For frontend, flags determine whether users see React or Backbone components. For backend, flags control which service handles specific API endpoints. This allows us to test in production with a small percentage of users, rollback instantly without code deployment, gradually increase adoption based on metrics, and run A/B tests to compare performance.

Canary Releases: New versions are deployed to a small subset of infrastructure first. We start with 5% of servers receiving the new version while monitoring key metrics. If metrics remain healthy for 30 minutes, we increase to 25%. After another validation period, we expand to 50%, then 100%. At any stage showing issues, we automatically rollback to the previous version.

Rollback Strategy: We maintain multiple rollback options at different speeds. Immediate rollback (under 5 minutes) uses feature flags to switch back to old behavior. Quick rollback (under 30 minutes) reverts the most recent deployment. Full rollback (under 2 hours) includes database rollback if schema changes occurred. All rollbacks are automated and triggered by monitoring alerts or manual intervention.

Parallel Running Systems: Both legacy and modern systems run simultaneously during migration. Play Framework and NestJS both serve production traffic on different endpoints. Backbone and React coexist in the same application. Database is shared between both backends. Monitoring tracks both systems independently. This allows real-time performance comparison, gradual user migration, safe validation of new system, and zero-downtime transitions.

Observability: Comprehensive monitoring ensures we detect issues immediately. Distributed tracing tracks requests across both systems with correlation IDs. Structured logging provides detailed context for debugging. Real-time metrics dashboards show performance by system. Automated alerts notify teams of anomalies. User session tracking helps identify migration impact.

Key Observability Metrics: Request success rate per system, response time percentiles (p50, p95, p99), error rates and types, database query performance, memory and CPU utilization, active user sessions, and feature adoption rates.

Risk Analysis

Risk 1: Hidden Coupling in Backbone Code

Description: Backbone applications often have implicit dependencies and global state that isn't immediately visible. Components may rely on global variables, event bus patterns, or undocumented initialization order.

Impact: High severity. Migrating one component might break another due to hidden dependencies. Global state mutations from Backbone could corrupt React component state. Timing issues could cause race conditions during initialization.

Mitigation Strategies: Conduct thorough code audit to map all dependencies. Document all global variables and their usage. Implement comprehensive integration tests covering component interactions. Use TypeScript to make implicit dependencies explicit. Create a dependency graph showing component relationships. Migrate interconnected components together as a unit. Add runtime checks for global state consistency.

Risk 2: Undocumented Play Controllers

Description: Legacy Play Framework code may lack documentation. Business logic could be scattered across controllers without clear patterns. Edge cases and error handling may only exist in code without documentation.

Impact: Medium to High severity. Developers might miss critical business rules during migration. Incomplete feature parity between old and new systems. Customer-facing bugs from missing edge case handling. Inconsistent error responses across systems.

Mitigation Strategies: Create comprehensive API documentation before migration starts. Add integration tests that verify all endpoints and edge cases. Pair experienced developers with new team members during migration. Document discovered business rules immediately. Use contract testing to ensure API compatibility. Maintain parallel testing environment comparing old and new responses.

Risk 3: Database Access Conflicts

Description: Two frameworks accessing the same database simultaneously can cause conflicts. Concurrent writes to the same records create race conditions. Different transaction isolation levels between JPA and TypeORM. Schema migration coordination between two systems. Connection pool exhaustion from double the connections.

Impact: High severity and critical. Data corruption from concurrent modifications. Deadlocks causing request failures. Performance degradation from lock contention. Database outages from connection exhaustion. Lost updates and inconsistent data state.

Mitigation Strategies: Implement clear endpoint ownership - only one system writes each endpoint. Use database-level row locking for critical operations. Monitor connection pool usage closely with alerts. Coordinate schema migrations between both systems carefully. Add optimistic locking with version columns. Implement retry logic with exponential backoff. Use database transaction logs to audit all changes.

Risk 4: Performance Regressions

Description: New system might be slower than legacy system. React may have higher memory usage than Backbone. NestJS could have different performance characteristics than Play. Additional network latency from routing between systems. Unoptimized queries in new ORM.

Impact: Medium severity but high visibility. Slower page loads affecting user experience. Increased infrastructure costs from higher resource usage. Customer complaints about application responsiveness. Reduced system throughput affecting business operations.

Mitigation Strategies: Establish performance baselines before migration. Load test each migrated component before production deployment. Profile both systems in production environment. Optimize database queries in new system. Implement caching strategies in NestJS. Use performance budgets and enforce in CI/CD. Monitor real user metrics (Core Web Vitals). Plan capacity increases based on observed usage patterns.

Risk 5: Authentication Inconsistencies

Description: Different JWT implementations between Play and NestJS. Session management across two systems. Token refresh mechanisms may differ. Different security libraries with varying behaviors. Cookie handling differences between frameworks.

Impact: High severity and security-critical. Users logged out unexpectedly due to token incompatibility. Security vulnerabilities from inconsistent validation. Session hijacking from implementation differences. Compliance issues with authentication standards. User frustration from repeated login prompts.

Mitigation Strategies: Use shared authentication configuration across both systems. Implement comprehensive auth integration tests. Validate JWT tokens against same secret and issuer. Document exact token format and validation rules. Use third-party auth service (Auth0, Okta) for consistency. Monitor authentication failure rates closely. Implement gradual rollout of auth changes. Maintain audit logs of all authentication events.

Risk 6: Team Knowledge and Training

Description: Team may lack experience with React, NestJS, or TypeScript. Different mental models between Java and TypeScript. Learning curve affecting development velocity. Knowledge silos where only few understand new stack.

Impact: Medium severity affecting timeline. Slower development during learning phase. Increased bug rates from unfamiliarity. Team frustration and potential turnover. Delayed project timeline. Reduced code quality initially.

Mitigation Strategies: Provide comprehensive training before migration starts. Pair experienced developers with learners. Schedule dedicated learning time each week. Create internal documentation and coding standards. Conduct regular code reviews focused on teaching. Build simple proof-of-concept projects for practice. Celebrate learning milestones. Budget additional time for learning curve.

Risk 7: Scope Creep and Over-Engineering

Description: Temptation to improve everything during migration. Adding features not in original system. Implementing perfect architecture instead of iterative. Migrating more than necessary in each phase.

Impact: Medium severity affecting timeline and cost. Project timeline extends significantly. Budget overruns from additional work. Team burnout from expanding scope. Core migration objectives delayed. Risk of never completing migration.

Mitigation Strategies: Define clear scope for each migration phase. Follow "lift and shift" principle first - move code, improve later. Track scope changes formally through change requests. Prioritize migration over new features. Set hard deadlines for each phase. Regular stakeholder alignment on priorities. Defer improvements to post-migration backlog.

Part 5: Code Review Discussion - Prepared Responses

Frontend Questions

Q1: Why did you choose this React integration approach?

Answer: We chose the Bridge pattern for React integration because it provides clear separation of concerns while maintaining a single source of truth. This approach minimizes code changes to existing Backbone application, allows gradual migration with each component independently testable, maintains business continuity with zero downtime, and provides instant rollback capability by simply unmounting React components.

The key advantage is that Backbone and React never directly communicate. The bridge translates between their different paradigms - Backbone's event-driven model and React's state-driven model. This isolation prevents the frameworks from conflicting and makes testing much easier.

Q2: How would you prevent Backbone and React state from diverging?

Answer: We prevent state divergence through strict architectural rules and technical implementation. First, Backbone models are the single source of truth - React never maintains independent state. Second, all data mutations go through Backbone models exclusively. Third, React subscribes to Backbone changes via the observable bridge pattern. Fourth, the bridge automatically propagates all changes from Backbone to React. Fifth, React state is derived from Backbone, never independently set.

Technical safeguards include integration tests that verify state consistency, runtime assertions checking state matches between systems, monitoring for divergence in production, and code reviews enforcing these patterns. If divergence occurs, automated tests would fail before deployment.

Q3: How would you migrate 500 Backbone views?

Answer: Migrating 500 views requires a systematic, prioritized approach. First, we analyze and categorize all views by complexity, usage frequency, business criticality, and dependencies. Second, we create a migration priority matrix ranking views for migration order. Third, we identify common patterns and create reusable React components. Fourth, we build a component library reducing duplication. Fifth, we migrate high-value views first for maximum impact.

The strategy would be: Start with independent views with no dependencies. Migrate frequently-used views for high user impact. Address business-critical views with extra testing. Handle complex views in smaller sub-components. Automate repetitive migrations where possible using codemods. Target 5-10 views per sprint with full testing. Run both versions in parallel during transition. Deprecate old views only after new ones are validated. Throughout, maintain comprehensive documentation and regular team knowledge sharing.

Q4: Would you introduce Redux/Zustand/Context immediately? Why?

Answer: No, I would not introduce Redux or similar state management immediately. The reason is that adding Redux while Backbone still exists creates three competing state systems (Backbone, React local state, Redux), which increases complexity significantly.

The correct phased approach is: Phase 1 uses Backbone models as the state source through the bridge. Phase 2 migrates to React with component-level useState for simple cases. Phase 3 introduces Context API for shared state across React components. Phase 4 evaluates Redux/Zustand only when Backbone is completely removed.

This approach minimizes complexity at each stage, maintains clear single source of truth, reduces cognitive load on team, allows proper evaluation of state management needs, and avoids premature optimization. Redux adds significant boilerplate and complexity that's only justified when you actually need its features like time-travel debugging, complex state logic, or large team coordination.

Backend Questions

Q1: Would you put NestJS behind the Play application or beside it?

Answer: I would put NestJS beside Play, not behind it, for several important reasons. Putting NestJS behind Play would force all requests through Play first, creating Play as a single point of failure, adding unnecessary latency from the proxy layer, and requiring significant Play code changes to implement routing logic.

Running beside offers clear advantages: Both systems are independent and can fail independently. Requests go directly to appropriate backend with no proxy overhead. Frontend or API gateway handles simple routing. Either system can be restarted without affecting the other. Both systems can scale independently based on load. Performance monitoring is simpler with direct access. Rollback is immediate by changing routing configuration.

The trade-off is slightly more complex routing logic in frontend or API gateway, but this is minimal compared to the benefits of independence and flexibility.

Q2: Would NestJS access the database directly?

Answer: Yes, NestJS should access the database directly using TypeORM. Indirect access through Play would create several problems: Play becomes a bottleneck for all database operations, double network hop adds latency, Play controllers must be maintained for database proxy, and tight coupling between systems makes independent deployment impossible.

Direct database access provides benefits: Both systems access database equally, NestJS can use TypeORM features fully, query optimization is straightforward, transaction management is simpler, and systems remain decoupled.

The critical requirement is schema coordination. Both must use identical schemas with matching tables and columns. Migrations must be coordinated between both systems. Clear endpoint ownership prevents write conflicts. Database monitoring tracks both systems. Shared connection pool limits prevent exhaustion.

Q3: How would you migrate authentication?

Answer: Authentication migration follows a careful phased approach because it's security-critical. Phase 1 implements shared JWT with same secret key in both systems, identical token format and validation, and both systems accepting same tokens. Phase 2 gradually migrates login endpoints to NestJS while maintaining existing tokens, running both login endpoints in parallel, and monitoring authentication metrics closely. Phase 3 consolidates to NestJS authentication with Play validating only, routing all new logins to NestJS, and planning final cutover. Phase 4 removes Play authentication completely once all users migrated and monitoring confirms stability.

Key safeguards include comprehensive auth testing, gradual rollout with feature flags, monitoring authentication failure rates, maintaining audit logs, having immediate rollback capability, and communicating changes to users if needed.

Q4: How would you prevent API contract drift?

Answer: Preventing API contract drift requires multiple technical and process controls. First, define OpenAPI/Swagger specifications for all endpoints as the source of truth. Second, implement contract testing comparing both implementations. Third, use automated tests validating request and response formats. Fourth, implement schema validation on both backends. Fifth, monitor API usage patterns in production.

Process controls include code review requirements checking contract compliance, documentation updates with every change, versioning APIs when breaking changes needed, and regular API contract reviews with frontend team. Additional safeguards are automated alerts on contract violations, integration tests calling both backends with same data, shadowing traffic to compare responses, and maintaining changelog of all API modifications.

Architecture Questions

Q1: What would you migrate first and why?

Answer: I would migrate the task list view first for frontend, and POST /api/tasks first for backend. The reasoning is strategic and practical.

For frontend, the task list is display-only with no complex user input, high visibility for immediate impact, relatively simple component structure, independent of other views, and provides learning opportunity with low risk. This choice delivers value quickly while building team confidence.

For backend, POST operations create new data avoiding conflicts with reads, clear ownership boundary with single writer, easy to validate and test, demonstrable progress to stakeholders, and low risk since failures only affect new data.

This approach follows the principle of starting with high-value, low-risk items to build momentum and prove the migration pattern before tackling complex components.

Q2: What would you explicitly avoid changing during migration?

Answer: Several things should remain unchanged to minimize risk and maintain focus.

Do not change the database schema except for absolute necessities. The existing schema works and both systems depend on it. Schema changes during migration multiply risk significantly.

Do not change API contracts unless absolutely required. Existing clients and frontend depend on current contracts. Breaking changes during migration create chaos and confusion.

Do not refactor working business logic while migrating. Separate migration from improvement. Refactoring introduces bugs that are hard to distinguish from migration issues.

Do not change deployment infrastructure simultaneously. Keep existing CI/CD, hosting, and monitoring. Infrastructure changes are separate projects with their own risks.

Do not add new features during core migration phases. Feature development and migration should be separate tracks to maintain clear priorities.

The principle is: migrate first, improve later. Mixing migration with improvements makes rollback impossible and debugging extremely difficult.

Q3: How do you measure migration success?

Answer: Success is measured across multiple dimensions with specific metrics.

Technical success metrics include zero production incidents from migration, response times maintained or improved, error rates below baseline, test coverage above 80%, and zero security regressions.

Migration progress metrics track percentage of endpoints migrated, percentage of frontend components in React, code coverage in new tests, legacy code removal rate, and deployment frequency maintenance.

Business success metrics include zero customer complaints about performance, feature velocity maintained, no missed releases, customer satisfaction maintained, and support ticket volume unchanged.

Team success metrics show developer satisfaction improved, onboarding time reduced, code review time reduced, and knowledge distribution improved.

The overall success indicator is delivering modern stack with zero business disruption. If customers don't notice the migration but developers are more productive, we've succeeded.

Q4: When would you stop migrating and reconsider?

Answer: Several signals would trigger reconsideration of the migration approach.

If error rates increase by more than 10% consistently, we stop and investigate root causes. The migration approach may be fundamentally flawed.

If performance degrades by more than 20% and optimization doesn't help, the new stack might not be suitable. Technical choices need reevaluation.

If team velocity decreases by more than 30% for more than two sprints, the approach is too complex. We need simpler migration patterns or better training.

If customer complaints increase significantly, business impact is too high. We pause and address concerns before proceeding.

If cost increases by more than 50% without clear benefits, the ROI isn't there. Economic justification needs reassessment.

If three consecutive rollbacks occur, the pattern isn't working. We need fundamental strategy changes, not just tactical adjustments.

The key is distinguishing temporary learning curve issues from fundamental problems. Temporary slowdowns are expected; sustained problems indicate deeper issues requiring strategy change.

Trade-offs Analysis

Trade-off 1: Frontend-Based Routing vs API Gateway

Decision: Frontend-based routing
Rationale: Simpler for development phase, no infrastructure dependencies, easy to test locally, transparent request flow, instant rollback capability.
Cost: Frontend needs to know about both backends, slightly more complex client code, not suitable for long-term production.
Future: Will migrate to API Gateway once migration stabilizes.

Trade-off 2: Shared Database vs Separate Databases

Decision: Shared database
Rationale: Eliminates data synchronization complexity, both systems see consistent data immediately, simpler transaction management, no eventual consistency issues.
Cost: Requires schema coordination, potential for write conflicts, connection pool sharing, migration coordination needed.
Mitigation: Clear endpoint ownership, coordinated migrations, monitoring, eventual microservices split with separate databases.

Trade-off 3: Bridge Pattern vs Complete Rewrite

Decision: Bridge pattern for incremental migration
Rationale: Zero downtime migration, reduced risk, continuous delivery maintained, gradual team learning, rollback capability.
Cost: Temporary code complexity, bridge maintenance overhead, dual system operation costs.
Future: Bridge removed once Backbone fully replaced, cleaner architecture emerges.

Trade-off 4: TypeScript Strict Mode Immediately vs Gradually

Decision: Strict mode from day one
Rationale: Better catch errors early, enforce best practices, easier maintenance long-term, team learns proper TypeScript.
Cost: Slower initial development, steeper learning curve, more boilerplate initially.
Benefit: Significantly fewer runtime errors, better IDE support, easier refactoring, improved code quality.

Trade-off 5: Microservices Now vs Monolith First

Decision: Monolith first (NestJS as single application)
Rationale: Simpler deployment, easier debugging, lower operational complexity, faster development.
Cost: Less independent scalability, tighter coupling initially.
Future: Split into microservices once system stabilizes and boundaries clear, after gaining operational experience.

Architecture Diagrams

Current Hybrid Architecture

Target Architecture

Request Flow - Current State

Data Flow - Frontend

```
User Interaction
       ↓
   React Component
       ↓
   Bridge Method Call
       ↓
   Backbone Model
       ↓
   HTTP Request to API
       ↓
   Backend Response
       ↓
   Backbone Collection Event
       ↓
   Bridge Notification
       ↓
   React setState
       ↓
   Component Re-render
       ↓
   Updated UI
```

Setup Instructions

Prerequisites

Required Software:

- Java 17 JDK
- SBT 1.9.7 or higher
- Node.js 22.x or higher
- npm 10.x or higher
- Git
- PostgreSQL 14+ (for production)

Optional Tools:

- Docker and Docker Compose
- Visual Studio Code or IntelliJ IDEA
- Postman or similar API testing tool

Initial Setup

Step 1: Clone Repository

```
git clone https://github.com/ahmedsaidani1/dev-assessment.git
cd dev-assessment
```

Step 2: Install Play Framework Dependencies

```
sbt update
```

Step 3: Install NestJS Dependencies

```
cd nestjs-api
npm install
cd ..
```

Running the Application

Terminal 1: Start Play Framework

```
sbt run
```

Application will start on http://localhost:9000

Terminal 2: Start NestJS

```
cd nestjs-api
npm run start:dev
```

Application will start on http://localhost:3000

Access the Application:
Open browser to http://localhost:9000

Verification Steps

1. Check Play Framework Health
Open http://localhost:9000 in browser. You should see the task management interface.

2. Check NestJS Health
Open http://localhost:3000/health or run:

```
curl http://localhost:3000/health
```

Expected response: JSON with status "healthy"

3. Test Task Creation

- Fill in task form on main page
- Click "Create Task"
- Check browser console (F12) for logs
- Should see "[Backbone] Routing POST request to NestJS backend"
- Task should appear in list below

4. Test Task Operations

- View tasks: Should load automatically
- Update status: Use dropdown on task
- Delete task: Click delete button

Running Tests

Play Framework Tests:

```
sbt test
```

NestJS Tests:

```
cd nestjs-api
npm test
```

Integration Tests:
Open browser developer console and verify logs show proper routing between systems.

Troubleshooting

Issue: Port Already in Use

If port 9000 is taken:

```
# Find process
netstat -ano | findstr :9000
# Kill process (replace PID)
taskkill /F /PID <PID>
```

If port 3000 is taken:

```
# Find process
netstat -ano | findstr :3000
# Kill process (replace PID)
taskkill /F /PID <PID>
```

Issue: SBT Compilation Errors

Clear SBT cache:

```
sbt clean
sbt compile
```

Issue: npm Dependencies Not Installing

Clear npm cache:

```
cd nestjs-api
rm -rf node_modules
rm package-lock.json
npm install
```

Issue: Application Not Loading

Check that Java 17 is being used:

```
java -version
```

Verify Node.js version:

```
node --version
```

Issue: CORS Errors in Browser

Verify NestJS is running on port 3000 and CORS is configured to allow localhost:9000.

Database Access

H2 Console (Development):
Access at http://localhost:9000/db/console

Connection Settings:

- JDBC URL: jdbc:h2:mem:play
- User: sa
- Password: (empty)

Environment Configuration

Play Framework (conf/application.conf):

```
db.default.driver=org.h2.Driver
db.default.url="jdbc:h2:mem:play"
```

NestJS (nestjs-api/src/app.module.ts):

```
TypeORM configured for SQLite in-memory
Production will use PostgreSQL
```

Development Workflow

1. Make Changes:

- Frontend: Edit files in public/javascripts/
- Backend Play: Edit files in app/
- Backend NestJS: Edit files in nestjs-api/src/

2. Hot Reload:

- Play: Automatically reloads on file changes
- NestJS: Automatically reloads with npm run start:dev
- Frontend: Refresh browser

3. Commit Changes:

```
git add .
git commit -m "Description of changes"
git push origin main
```

Production Deployment

Using Docker Compose:

```
docker-compose build
docker-compose up -d
```

Using Kubernetes:

```
kubectl apply -f kubernetes/
```

Refer to PART3_IMPLEMENTATION_GUIDE.md for detailed deployment instructions.

End of Complete Migration Design Document

This document provides comprehensive guidance for migrating from legacy to modern architecture with minimal risk and zero downtime.